

Practical 2

Aim

To develop a C program using the system calls `open()`, `read()`, `write()`, and `close()` to copy the contents of one file to another and to understand the transition of control between user space and kernel space during file operations.

Requirements

- macOS Terminal / Linux Terminal
- C Compiler (GCC/Clang)
- Text editor (`nano`)
- Input file: `sample.txt`

Procedure

1. A working directory named `oss_practical2` was created.
2. A source file named `sample.txt` was created and sample content was added to it.
3. A C program named `filecopy.c` was created.
4. The `open()` system call was used to open the source file in read-only mode.
5. The destination file `copy.txt` was created using the `open()` system call with write and create permissions.
6. The `read()` system call was used to read the contents of the source file into a buffer.
7. The `write()` system call was used to write the buffer contents into the destination file.
8. The `close()` system call was used to close both files after the copying operation was completed.
9. The program was compiled and executed successfully.
10. The contents of `copy.txt` were displayed to verify that the file was copied correctly.

Source File

The file `sample.txt` contains the following data:

This is the source file for OSS Practical 2.

The program demonstrates file operations using system calls.

`open()`, `read()`, `write()`, and `close()` are used.

```
[sahasra@Mac oss_practical2 % nano sample.txt
[sahasra@Mac oss_practical2 % cat sample.txt
This is the source file for OSS Practical 2.
The program demonstrates file operations using system calls.
open(), read(), write(), and close() are used.
```

C Program

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#define BUFFER_SIZE 1024

int main() {
    int source, destination;
    ssize_t bytesRead, bytesWritten;
    char buffer[BUFFER_SIZE];

    source = open("sample.txt", O_RDONLY);

    if (source == -1) {
        perror("Error opening source file");
        exit(1);
    }
    destination = open("copy.txt",
                      O_WRONLY | O_CREAT | O_TRUNC,
                      0644);

    if (destination == -1) {
        perror("Error opening destination file");
        close(source);
        exit(1);
    }

    while ((bytesRead = read(source, buffer, BUFFER_SIZE)) > 0) {
        bytesWritten = write(destination, buffer, bytesRead);

        if (bytesWritten != bytesRead) {
            perror("Error writing to destination file");
            close(source);
            close(destination);
            exit(1);
        }
    }

    if (bytesRead == -1) {
        perror("Error reading source file");
    }

    close(source);
    close(destination);

    printf("File copied successfully!\n");
    return 0;
}
```

Compilation and Execution

The program was compiled using the following command:

```
gcc filecopy.c -o filecopy
```

The executable program was run using:

```
./filecopy
```

The output obtained was:

```
File copied successfully!
```

```
sahasra@Mac oss_practical2 % ./filecopy
File copied successfully! _
```

Verification of Copied File

The destination file was verified using the following command:

```
cat copy.txt
```

The contents displayed in `copy.txt` were the same as the contents of `sample.txt`.

```
[sahasra@Mac oss_practical2 % cat copy.txt
This is the source file for OSS Practical 2.
The program demonstrates file operations using system calls.
open(), read(), write(), and close() are used.
```

Explanation of System Calls

open()

The `open()` system call is used to open an existing file or create a new file. In this program, `sample.txt` is opened in read-only mode, while `copy.txt` is opened for writing and is created if it does not already exist.

read()

The `read()` system call transfers data from the source file into a buffer in the program's memory. It continues reading until the complete contents of the file have been processed.

write()

The `write()` system call transfers the data stored in the buffer to the destination file.

close()

The `close()` system call releases the file descriptors associated with the source and destination files after the file operations are completed.

User Space and Kernel Space Transition

The C program initially executes in **user space**. When the program requires an operation such as opening, reading, writing, or closing a file, it cannot directly access the file system hardware or kernel-managed resources.

Therefore, the program makes a system call.

The sequence of execution is as follows:

User Program → System Call → Kernel Space → File System Operation → Return to User Space

1. The program executes in user space.
2. The program calls `open()`, `read()`, `write()`, or `close()`.
3. Control transfers from user space to kernel space through a system call.
4. The operating system kernel performs the requested file operation.
5. The result of the operation is returned to the user program.
6. The program continues execution in user space.

Thus, system calls provide a controlled interface through which user programs can request services from the operating system kernel.

Result

The C program was successfully executed using the system calls `open()`, `read()`, `write()`, and `close()`. The contents of `sample.txt` were successfully copied to `copy.txt`, demonstrating file operations and the transition of control between user space and kernel space.